TfL Analytics API

Overview

API for user routes, favourite stopss, incidents, and TfL analytics. Completed as a coursework requirement for COMP3011 Web Services and Web Data.

Version information

Version : v1

URI scheme

Host : 127.0.0.1:8000
BasePath : /api
Schemes : HTTP

Consumes

- `application/json`

Produces

- `application/json`

Security

Basic

Type : basic

Paths

Compute average headway for a line

```
GET /analytics/lines/{line_id}/headway/
```

Description

Computes the average headway (time between consecutive services) for a line.

Parameters

Type	Name	Schema
Path	**line_id** *required*	string

Responses

HTTP Code	Description	Schema
200	Successfully computed average headway for the line	No Content
404	Line not found or has insufficient arrival data	No Content

Tags

- Analytics - Lines

Example HTTP response

Response 200

```
{
  "line_id" : "victoria",
  "average_headway_seconds" : 180.0
}
```

Response 404

```
{
  "detail" : "Line 'southern' not found or has insufficient arrival data."
}
```

Line status based on user incidents

```
GET /analytics/lines/{line_id}/incident-status/
```

Description

Computes the operational status of a line using recent user-reported incidents. The status is categorized based on the severity and number of incidents reported in the last 30 minutes.

Parameters

Type	Name	Schema
Path	**line_id** *required*	string

Responses

HTTP Code	Description	Schema
200	Incident-based line status	No Content
400	Error: Bad Request - Invalid Line ID	No Content

Tags

- Analytics - Lines

Example HTTP response

Response 200

```
{
  "line_id" : "victoria",
  "status" : "Moderate delays",
  "score" : 5
}
```

Response 400

```
{
  "error" : "Invalid Line ID"
}
```

Compute average wait time for a stop

```
GET /analytics/stops/{stop_id}/average-wait/
```

Description

Computes the average wait time (in seconds) for a specific stop based on current arrival data for the station (i.e., irrespective of the line, calculates the average wait).

Parameters

Type	Name	Schema
Path	**stop_id** *required*	string

Responses

HTTP Code	Description	Schema
200	Average wait time for the stop	No Content
400	Invalid stop ID	No Content
404	Stop has no arrival data	No Content

Tags

- Analytics - Stops

Example HTTP response

Response 200

```
{
  "stop_id" : "940GZZLUBST",
  "average_wait_seconds" : 142.5
}
```

Response 400

```
{
  "error" : "Invalid stop ID"
}
```

Response 404

```
{
  "detail" : "Stop 'HUBZMG' not found or has no arrival data."
}
```

Stop status based on user incidents

```
GET /analytics/stops/{stop_id}/incident-status/
```

Description

Computes the operational status of a stop using recent user-reported incidents. The status is categorized based on the severity and number of incidents reported in the last 30 minutes.

Parameters

Type	Name	Schema
Path	**stop_id** *required*	string

Responses

HTTP Code	Description	Schema
200	Incident-based stop status	No Content
400	Error: Bad Request - Invalid Stop ID	No Content

Tags

- Analytics - Stops

Example HTTP response

Response 200

```
{
  "stop_id" : "940GZZLUBST",
  "status" : "Moderate issues",
  "score" : 5
}
```

Response 400

```
{
  "error" : "Invalid Stop ID"
}
```

Report a user incident

```
POST /incidents/
```

Description

Creates a new incident report for a stop and line.

Parameters

Type	Name	Schema
Body	**data** *required*	UserIncident

Responses

HTTP Code	Description	Schema
201	Incident created	No Content
400	Invalid input data	No Content

Tags

- User Incidents

Example HTTP response

Response 201

```
{
  "id" : 3,
  "user" : "John Doe",
  "stop" : "Whitechapel (HUBZWL)",
  "line" : "Elizabeth line",
  "description" : "Severe delays, platform overcrowded",
  "severity" : 4
}
```

Response 400

```
{
  "line" : "Line ID does not exist."
}
```

Retrieve all user-reported incidents

```
GET /incidents/
```

Description

Retrieves all user-reported incidents.

Responses

HTTP Code	Description	Schema
200	List of user incidents. Returns empty list if no issues reported.	No Content

Tags

- User Incidents

Example HTTP response

Response 200

```
[ {
  "id" : 1,
  "user" : "John Doe",
  "stop" : "Whitechapel (HUBZWL)",
  "line" : "Elizabeth line",
  "description" : "Severe delays, platform overcrowded",
  "severity" : 4,
  "created_at" : "2026-03-12T17:30:00Z"
}, {
  "id" : 2,
  "user" : "Jane Smith",
  "stop" : "Liverpool Street (HUBZLST)",
  "line" : "Central line",
  "description" : "Minor delays, train at full capacity",
  "severity" : 2,
  "created_at" : "2026-03-12T16:45:00Z"
} ]
```

Retrieve a specific incident

```
GET /incidents/{id}/
```

Description

Retrieve a specific incident from a given Incident ID. Example: ID = 1

Parameters

Type	Name	Description	Schema
Path	**id** *required*	A unique integer value identifying this user incident.	integer

Responses

HTTP Code	Description	Schema
200	Incident details	No Content
404	Incident not found	No Content

Tags

- User Incidents

Example HTTP response

Response 200

```
{
  "id" : 1,
  "user" : "John Doe",
  "stop" : "Whitechapel (HUBZWL)",
  "line" : "Elizabeth line",
  "description" : "Severe delays, platform overcrowded",
  "severity" : 4,
  "created_at" : "2026-03-12T17:30:00Z"
}
```

Response 404

```
{
  "detail" : "No UserIncident matches the given query."
}
```

Update a specific incident

```
PUT /incidents/{id}/
```

Description

Update a specific incident using a given Incident ID

Parameters

Type	Name	Description	Schema
Path	**id** *required*	A unique integer value identifying this user incident.	integer
Body	**data** *required*		UserIncident

Responses

HTTP Code	Description	Schema
200	Incident details after update.	No Content
400	Error: Bad Request (Invalid input data - full request body required)	No Content
404	Incident not found	No Content

Tags

- User Incidents

Example HTTP response

Response 200

```
{
  "id" : 1,
  "user" : "John Doe",
  "stop" : "Whitechapel (HUBZWL)",
  "line" : "Elizabeth line",
  "description" : "Severe delays, platform overcrowded",
  "severity" : 4,
  "created_at" : "2026-03-12T17:30:00Z"
}
```

Response 400

```
{
  "stop" : [ "Stop ID is required." ]
}
```

Response 404

```
{
  "detail" : "No UserIncident matches the given query."
}
```

Delete a specific incident

```
DELETE /incidents/{id}/
```

Description

Delete a specific incident via a given Incident ID

Parameters

Type	Name	Description	Schema
Path	**id** *required*	A unique integer value identifying this user incident.	integer

Responses

HTTP Code	Description	Schema
204	Incident deleted successfully	No Content
404	Incident not found	No Content

Tags

- User Incidents

Example HTTP response

Response 404

```
{
  "detail" : "No UserIncident matches the given query."
}
```

Partially update a specific incident

```
PATCH /incidents/{id}/
```

Description

Partially update a specific incident using a given Incident ID. Only the updated fields need to be provided in the request body.

Parameters

Type	Name	Description	Schema
Path	**id** *required*	A unique integer value identifying this user incident.	integer
Body	**data** *required*		UserIncident

Responses

HTTP Code	Description	Schema
200	Incident details after partial update.	No Content
404	Incident not found	No Content

Tags

- User Incidents

Example HTTP response

Response 200

```
{
  "id" : 1,
  "user" : "John Doe",
  "stop" : "Whitechapel (HUBZWL)",
  "line" : "Elizabeth line",
  "description" : "Severe delays, platform overcrowded",
  "severity" : 4,
  "created_at" : "2026-03-12T17:30:00Z"
}
```

Response 404

```
{
  "detail" : "No UserIncident matches the given query."
```

```
}
```

Retrieve all lines in the database

```
GET /lines/
```

Description

Retrieve all lines in the database.

Responses

HTTP Code	Description	Schema
200	List of lines	No Content

Tags

- Lines

Example HTTP response

Response 200

```
[ {
  "line_id" : "bakerloo",
  "name" : "Bakerloo",
  "mode" : "tube"
}, {
  "line_id" : "central",
  "name" : "Central",
  "mode" : "tube"
}, {
  "line_id" : "circle",
  "name" : "Circle",
  "mode" : "tube"
} ]
```

Add a new favourite route

```
POST /routes/
```

Description

Adds a new route for a user, including origin stop, destination stop, and line.

Parameters

Type	Name	Schema
Body	**data** *required*	UserRoute

Responses

HTTP Code	Description	Schema
201	Route created successfully	No Content
400	Invalid input data	No Content

Tags

- User Routes

Example HTTP response

Response 201

```
{
  "id" : 12,
  "user" : "John Doe",
  "from_stop" : "Farringdon (HUBZFD)",
  "to_stop" : "Whitechapel (HUBZWL)",
  "line" : "Elizabeth line"
}
```

Response 400

```
{
  "from_stop" : "Stop ID does not exist."
}
```

Retrieve all user routes

```
GET /routes/
```

Description

Retrieve all user-added/favourite routes.

Responses

HTTP Code	Description	Schema
200	List of user routes	No Content

Tags

- User Routes

Example HTTP response

Response 200

```
[ {
  "id" : 1,
  "user" : "John Doe",
  "from_stop" : "Farringdon (HUBZFD)",
  "to_stop" : "Whitechapel (HUBZWL)",
  "line" : "Elizabeth line"
}, {
  "id" : 2,
  "user" : "Jane Smith",
  "from_stop" : "Ealing Broadway (HUBEAL)",
  "to_stop" : "Liverpool Street (HUBZLST)",
  "line" : "Central line"
} ]
```

Retrieve a user route

```
GET /routes/{id}/
```

Description

Retrieve a specific user route by its ID.

Parameters

Type	Name	Description	Schema
Path	**id** *required*	A unique integer value identifying this user route.	integer

Responses

HTTP Code	Description	Schema
200	User route details	No Content
404	Route not found	No Content

Tags

- User Routes

Example HTTP response

Response 200

```
{
  "id" : 1,
  "user" : "John Doe",
  "from_stop" : "Farringdon (HUBZFD)",
  "to_stop" : "Whitechapel (HUBZWL)",
  "line" : "Elizabeth line"
}
```

Response 404

```
{
  "error" : "User route not found"
}
```

Update a user route

```
PUT /routes/{id}/
```

Description

Update a specific user route

Parameters

Type	Name	Description	Schema
Path	**id** *required*	A unique integer value identifying this user route.	integer

Type	Name	Description	Schema
Body	**data** *required*		UserRoute

Responses

HTTP Code	Description	Schema
200	Route updated successfully	No Content
400	Invalid input data	No Content
404	Route not found	No Content

Tags

- User Routes

Example HTTP response

Response 200

```
{
  "id" : 1,
  "user" : "John Doe",
  "from_stop" : "Farringdon (HUBZFD)",
  "to_stop" : "Whitechapel (HUBZWL)",
  "line" : "Elizabeth line"
}
```

Response 400

```
{
  "from_stop" : "Stop ID does not exist."
}
```

Response 404

```
{
  "detail" : "No UserRoute matches the given query."
}
```

Delete a user route

```
DELETE /routes/{id}/
```

Description

Delete a specific user route

Parameters

Type	Name	Description	Schema
Path	**id** *required*	A unique integer value identifying this user route.	integer

Responses

HTTP Code	Description	Schema
204	Route deleted successfully	No Content
404	Route not found	No Content

Tags

- User Routes

Example HTTP response

Response 404

```
{
  "detail" : "No UserRoute matches the given query."
}
```

Partially update a user route

```
PATCH /routes/{id}/
```

Description

Partially update a specific user route. Only the updated fields need to be provided in the request body.

Parameters

Type	Name	Description	Schema
Path	**id** *required*	A unique integer value identifying this user route.	integer
Body	**data** *required*		UserRoute

Responses

HTTP Code	Description	Schema
200	Route partially updated successfully	No Content
400	Invalid input data	No Content
404	Route not found	No Content

Tags

- User Routes

Example HTTP response

Response 200

```
{
  "id" : 1,
  "user" : "John Doe",
  "from_stop" : "Farringdon (HUBZFD)",
  "to_stop" : "Whitechapel (HUBZWL)",
  "line" : "Elizabeth line"
}
```

Response 400

```
{
  "from_stop" : "Stop ID does not exist."
}
```

Response 404

```
{
  "detail" : "No UserRoute matches the given query."
}
```

Add a favourite stop

```
POST /stations/
```

Description

Marks a stop as a favourite for a user.

Parameters

Type	Name	Schema
Body	**data** *required*	UserStation

Responses

HTTP Code	Description	Schema
201	Favourite stop added	No Content

Tags

- User Stops

Example HTTP response

Response 201

```
{
  "id" : 5,
  "user" : "John Doe",
  "stop" : "Farringdon (HUBZFD)"
}
```

Retrieve all favourite stops

```
GET /stations/
```

Description

Retrieve all favourite stops for all users

Responses

HTTP Code	Description	Schema
200	List of user favourite stops. Returns empty list if no favourite stops added.	No Content

Tags

- User Stops

Example HTTP response

Response 200

```
[ {
  "id" : 1,
  "user" : "John Doe",
  "stop" : "Farringdon (HUBZFD)"
}, {
  "id" : 2,
  "user" : "Jane Smith",
  "stop" : "Whitechapel (HUBZWL)"
} ]
```

Retrieve a specific favourite stop

```
GET /stations/{id}/
```

Description

Retrieve a specific favourite stop for a user

Parameters

Type	Name	Description	Schema
Path	**id** *required*	A unique integer value identifying this user station.	integer

Responses

HTTP Code	Description	Schema
200	Favourite stop details	No Content

HTTP Code	Description	Schema
404	Favourite stop not found	No Content

Tags

- User Stops

Example HTTP response

Response 200

```
{
  "id" : 1,
  "user" : "John Doe",
  "stop" : "Farringdon (HUBZFD)"
}
```

Response 404

```
{
  "detail" : "No UserStation matches the given query."
}
```

Delete a favourite stop

```
DELETE /stations/{id}/
```

Description

Delete a specific favourite stop for a user

Parameters

Type	Name	Description	Schema
Path	**id** *required*	A unique integer value identifying this user station.	integer

Responses

HTTP Code	Description	Schema
204	Favourite stop deleted successfully	No Content
404	Favourite stop not found	No Content

Tags

- User Stops

Example HTTP response

Response 404

```
{
  "detail" : "No UserStation matches the given query."
}
```

Retrieve all stops in the database

```
GET /stops/
```

Description

Retrieve all stops in the database. Supports filtering by mode (e.g. tube, elizabeth-line, dlr, overground, tram).

Parameters

Type	Name	Schema
Query	**mode** *optional*	string

Responses

HTTP Code	Description	Schema
200	List of stops as per filter. (Example filter: "elizabeth-line")	No Content

Tags

- Stops

Example HTTP response

Response 200

```
[ {
  "stop_id" : "HUBBDS",
  "name" : "Bond Street",
  "mode" : "elizabeth-line",
  "lat" : 51.513362,
  "lon" : -0.148795
}, {
  "stop_id" : "HUBCAW",
  "name" : "Canary Wharf",
  "mode" : "elizabeth-line",
  "lat" : 51.503734,
  "lon" : -0.019121
}, {
  "stop_id" : "HUBEAL",
  "name" : "Ealing Broadway",
  "mode" : "elizabeth-line",
  "lat" : 51.514993,
  "lon" : -0.302131
} ]
```

Retrieve a stop by ID

```
GET /stops/{stop_id}/
```

Description

Retrieve detailed information about a specific stop using its stop ID. Returns name, mode, and coordinates. Useful for validating stop IDs or displaying stop metadata.

Parameters

Type	Name	Schema
Path	**stop_id** *required*	string

Responses

HTTP Code	Description	Schema
200	Stop found	No Content
404	Stop not found	No Content

Tags

- Stops

Example HTTP response

Response 200

```
{
  "stop_id" : "HUBBDS",
  "name" : "Bond Street",
  "mode" : "elizabeth-line",
  "lat" : 51.513362,
  "lon" : -0.148795
}
```

Response 404

```
{
  "error" : "Stop ID not found"
}
```

Retrieve arrival predictions for a stop

```
GET /stops/{stop_id}/arrivals/
```

Description

Retrieves all previously saved arrival predictions for a stop. Use refresh = true to fetch and additionally return fresh data from TfL.

Parameters

Type	Name	Description	Schema
Path	**stop_id** *required*		string
Query	**refresh** *optional*	If true, fetch fresh arrival data from TfL before returning results.	boolean

Responses

HTTP Code	Description	Schema
200	List of arrival predictions	No Content
400	Error: Bad Request - Invalid Stop ID	No Content

Tags

- Arrivals

Example HTTP response

Response 200

```
[ {
  "id" : 2627,
  "direction" : "outbound",
  "destination_name" : "Hainault Underground Station",
  "predicted_time" : "2026-03-12T18:11:37Z",
  "time_to_station" : 1611,
  "recorded_at" : "2026-03-12T17:44:54.319715Z",
  "stop" : "940GZZLUNBP",
  "line" : "central"
}, {
  "id" : 2628,
  "direction" : "outbound",
  "destination_name" : "Hainault Underground Station",
  "predicted_time" : "2026-03-12T17:45:38Z",
  "time_to_station" : 52,
  "recorded_at" : "2026-03-12T17:44:54.323132Z",
  "stop" : "940GZZLUNBP",
  "line" : "central"
}, {
  "..." : "..."
} ]
```

Response 400

```
{
  "error" : "Invalid Stop ID"
}
```

Definitions

UserIncident

Name	Description	Schema
created_at *optional* *read-only*		string (date-time)
description *required*	Description of the incident **Length** : `1 - 200`	string
id *optional* *read-only*		integer
line *required*	Line ID affected by the incident **Length** : `1 - 20`	string
severity *required*	Severity level (1-5) **Minimum value** : `1` **Maximum value** : `5`	integer
stop *required*	Stop ID where the incident occurred **Length** : `1 - 20`	string
user *optional* *read-only*	**Minimum length** : `1`	string

UserRoute

Name	Description	Schema
from_stop *required*	Origin stop ID **Length** : `1 - 20`	string
id *optional* *read-only*		integer
line *required*	Line ID for the route **Length** : `1 - 20`	string
to_stop *required*	Destination stop ID **Length** : `1 - 20`	string
user *optional* *read-only*	**Minimum length** : `1`	string

UserStation

Name	Description	Schema
id *optional* *read-only*		integer
stop *required*	Stop ID to mark as favourite **Length** : `1 - 20`	string
user *optional* *read-only*	**Minimum length** : `1`	string